PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

In [1]:
```
#Importation de la librairie Pandas
import pandas as pd
```

1.2 - Chargement des fichiers Excel

In [2]:
```
#Importation du fichier population.csv
population = pd.read_csv('population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

In [3]:
```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(populatio
```

```
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

```
Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

In [4]:
```
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes

population.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1416 entries, 0 to 1415
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Zone    1416 non-null   object
 1   Année   1416 non-null   int64
 2   Valeur  1416 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 33.3+ KB
```

In [5]:
```
#Affichage les 5 premières lignes de la table
population.head()
```

Out[5]:

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

In [6]:
```
#Nous allons harmoniser les unités. Pour cela, nous avons décidé de multipli
#Multiplication de la colonne valeur par 1000
population["Valeur"]*=1000
```

In [7]:
```
#changement du nom de la colonne Valeur par Population
population.rename(columns={'Valeur':'Population'}, inplace=True)
```

In [8]:
```
#Affichage les 5 premières lignes de la table pour voir les modifications
population.head()
```

Out[8]:

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

In [9]:
```
#Afficher les dimensions du dataset
dispo_alimentaire.shape
```

Out[9]: (15605, 18)

In [10]:
```
#Consulter le nombre de colonnes
dispo_alimentaire.shape[1]
```

Out[10]: 18

In [11]:
```
#Affichage les 5 premières lignes de la table
dispo_alimentaire.head(20)
```

Out[11]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70
5	Afghanistan	Beurre, Ghee	animale	NaN	NaN	23.0	1.17
6	Afghanistan	Bière	vegetale	NaN	NaN	0.0	0.09
7	Afghanistan	Blé	vegetale	NaN	NaN	1369.0	160.23
8	Afghanistan	Boissons Alcooliques	vegetale	NaN	NaN	0.0	0.00
9	Afghanistan	Café	vegetale	NaN	NaN	0.0	0.00
10	Afghanistan	Coco (Incl Coprah)	vegetale	NaN	NaN	0.0	0.00
11	Afghanistan	Crème	animale	NaN	NaN	3.0	0.60
12	Afghanistan	Céréales, Autres	vegetale	NaN	NaN	0.0	0.00
13	Afghanistan	Dattes	vegetale	NaN	NaN	0.0	0.02
14	Afghanistan	Edulcorants Autres	vegetale	NaN	NaN	2.0	0.53
15	Afghanistan	Feve de Cacao	vegetale	NaN	NaN	0.0	0.02
16	Afghanistan	Fruits, Autres	vegetale	NaN	NaN	9.0	7.29
17	Afghanistan	Graines de coton	vegetale	NaN	NaN	NaN	NaN
18	Afghanistan	Graines de tournesol	vegetale	NaN	NaN	NaN	NaN
19	Afghanistan	Graisses Animales Crue	animale	NaN	NaN	12.0	0.49

In [12]:
```
#remplacement des NaN dans le dataset par des 0
dispo_alimentaire.fillna(0,inplace=True)
```

In [13]:
```
#multiplication de toutes les lignes contenant des milliers de tonnes en Kg

colonnes_tonnes_tokg = ['Aliments pour animaux', 'Disponibilité intérieure',
                        'Importations - Quantité', 'Nourriture', 'Pertes', '
                        'Semences', 'Traitement', 'Variation de stock', 'Aut

for el in colonnes_tonnes_tokg:
    dispo_alimentaire[el] *= 1000000
```

In [14]:
```
#Affichage les 5 premières lignes de la table
dispo_alimentaire.head()
```

Out[14]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	0.00
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	2.70

2.3 - Analyse exploratoire du fichier aide alimentaire

In [15]:
```
#Afficher les dimensions du dataset
aide_alimentaire.shape
```

Out[15]: (1475, 4)

In [16]:
```
#Consulter le nombre de colonnes
aide_alimentaire.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1475 entries, 0 to 1474
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Pays bénéficiaire  1475 non-null   object
 1   Année              1475 non-null   int64
 2   Produit            1475 non-null   object
 3   Valeur             1475 non-null   int64
dtypes: int64(2), object(2)
memory usage: 46.2+ KB
```

In [17]:
```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

Out[17]:

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

In [18]:
```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={"Pays bénéficiaire":"Zone"},inplace=True)
```

In [19]:
```
#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1
aide_alimentaire.rename(columns={"Valeur":"Aide_alimentaire"},inplace=True)

aide_alimentaire["Aide_alimentaire"]*=1000
```

In [20]:
```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

Out[20]:

	Zone	Année	Produit	Aide_alimentaire
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

In [21]:
```
#Afficher les dimensions du dataset
sous_nutrition.shape
```

Out[21]:
```
(1218, 3)
```

In [22]:
```
#Consulter le nombre de colonnes
sous_nutrition.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Zone    1218 non-null   object
 1   Année   1218 non-null   object
 2   Valeur  624 non-null    object
dtypes: object(3)
memory usage: 28.7+ KB
```

In [23]:
```
#Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

Out[23]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

In [24]:
```
#Conversion de la colonne sous nutrition en numérique
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'], errors='c
```

In [25]:
```
#Conversion de la colonne (avec l'argument errors=coerce qui permet de conve
#Puis remplacement des NaN en 0
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].fillna(0)
```

In [26]:
```
#changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition.rename(columns={"Valeur":"sous_nutrition"},inplace=True)
```

In [27]:
```
#Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition["sous_nutrition"]*=1000000
```

In [28]:
```
#Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

Out[28]:

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

In [29]:
```
# Il faut tout d'abord faire une jointure entre la table population et la ta
population_2017 = population[population["Année"]==2017].copy()
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018']
```

In [30]:
```
#Affichage du dataset
population_2017 = population_2017.rename(columns={'Zone': 'Pays'})
sous_nutrition_2017 = sous_nutrition_2017.rename(columns={'Zone': 'Pays'})

sous_nutrition_population = pd.merge(sous_nutrition_2017, population_2017, o
sous_nutrition_population
```

Out[30]:

	Pays	Année_x	sous_nutrition	Année_y	Population
0	Afghanistan	2016-2018	10500000.0	2017	36296113.0
1	Afrique du Sud	2016-2018	3100000.0	2017	57009756.0
2	Albanie	2016-2018	100000.0	2017	2884169.0
3	Algérie	2016-2018	1300000.0	2017	41389189.0
4	Allemagne	2016-2018	0.0	2017	82658409.0
...	...	...	...	...	...
198	Venezuela (République bolivarienne du)	2016-2018	8000000.0	2017	29402484.0
199	Viet Nam	2016-2018	6500000.0	2017	94600648.0
200	Yémen	2016-2018	0.0	2017	27834819.0
201	Zambie	2016-2018	0.0	2017	16853599.0
202	Zimbabwe	2016-2018	0.0	2017	14236595.0

203 rows × 5 columns

In [31]:
```
#Calcul et affichage du nombre de personnes en état de sous nutrition
sous_nutrition_population['Nombre en sous-nutrition'] = (sous_nutrition_popu
sous_nutrition_population.head()
```

Out[31]:

	Pays	Année_x	sous_nutrition	Année_y	Population	Nombre en sous-nutrition
0	Afghanistan	2016-2018	10500000.0	2017	36296113.0	3.811092e+12
1	Afrique du Sud	2016-2018	3100000.0	2017	57009756.0	1.767302e+12
2	Albanie	2016-2018	100000.0	2017	2884169.0	2.884169e+09
3	Algérie	2016-2018	1300000.0	2017	41389189.0	5.380595e+11
4	Allemagne	2016-2018	0.0	2017	82658409.0	0.000000e+00

In [32]:
```
((sous_nutrition_population["sous_nutrition"].sum()/sous_nutrition_populatio
```

Out[32]: 7.1011968332354165

3.2 - Nombre théorique de personne qui pourrait être nourries

In [33]:
```
#Combien mange en moyenne un être humain ? Source =>
kcal_annuelles_humain = 2500 * 365
```

In [34]:
```
#On commence par faire une jointure entre le data frame population et Dispo_
dispo_alimentaire = dispo_alimentaire.merge(population.loc[population['Année
                                            on='Zone')
```

In [92]:
```
#Affichage du nouveau dataframe
dispo_alimentaire.head(10)
```

Out[92]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	0.06
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	0.00
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	2.70
5	Afghanistan	Beurre, Ghee	animale	0.0	0.0	23.0	1.17
6	Afghanistan	Bière	vegetale	0.0	0.0	0.0	0.09
7	Afghanistan	Blé	vegetale	0.0	0.0	1369.0	160.23
8	Afghanistan	Boissons Alcooliques	vegetale	0.0	0.0	0.0	0.00
9	Afghanistan	Café	vegetale	0.0	0.0	0.0	0.00

In [36]:
```
#Création de la colonne dispo_kcal avec calcul des kcal disponibles mondiale
dispo_alimentaire['dispo_kcal'] = dispo_alimentaire['Disponibilité alimentai
dispo_alimentaire['dispo_kcal'].sum()
```

Out[36]: 7635429388975815.0

In [37]:
```
#Calcul du nombre d'humains pouvant être nourris
kcal_moy_annuel = 2500 * 365

humains_nourris= dispo_alimentaire['dispo_kcal'].sum() / kcal_moy_annuel
print(humains_nourris)
```

8367593850.9324

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

In [38]:
```
#Transfert des données avec les végétaux dans un nouveau dataframe
vegetaux_df = dispo_alimentaire[dispo_alimentaire['Origine'] == 'vegetale'].
```

In [39]:
```
#Calcul du nombre de kcal disponible pour les végétaux
vegetaux_df['dispo_kcal'].sum()
```

Out[39]: 6300178937197865.0

In [40]:
```
#Calcul du nombre d'humains pouvant être nourris avec les végétaux
humains_nourris= vegetaux_df['dispo_kcal'].sum() / kcal_moy_annuel
print(humains_nourris)
```

```
6904305684.6004
```

3.4 - Utilisation de la disponibilité intérieure

In [41]:
```
#Calcul de la disponibilité totale
dispo_int = dispo_alimentaire["Disponibilité intérieure"].sum()
print(dispo_int)
```

```
9733927000000.0
```

In [42]:
```
#création d'une boucle for pour afficher les différentes valeurs en fonction
for elt in ['Aliments pour animaux', 'Pertes', 'Nourriture', 'Semences', 'Tr
    print("Proportion de", elt, ":", "{:.2f}".format(dispo_alimentaire[elt].
```

```
Proportion de Aliments pour animaux : 13.23 %
Proportion de Pertes : 4.65 %
Proportion de Nourriture : 49.37 %
Proportion de Semences : 1.58 %
Proportion de Traitement : 22.45 %
Proportion de Autres Utilisations : 8.82 %
```

3.5 - Utilisation des céréales

In [43]:
```
#Création d'une liste avec toutes les variables
liste_cereales = ["Blé", "Riz (Eq Blanchi)", "Orge", "Maïs", "Seigle",
                  "Avoine", "Millet", "Sorgho", "Céréales, Autres"]
```

In [48]:
```
#Création d'un dataframe avec les informations uniquement pour ces céréales

cereales = dispo_alimentaire.loc[dispo_alimentaire['Produit'].isin(liste_cer
```

In [49]:
```
#Affichage de la proportion d'alimentation animale
cereales.head()
```

Out[49]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibil alimentai en quanti (k personr a
7	Afghanistan	Blé	vegetale	0.0	0.0	1369.0	160.
12	Afghanistan	Céréales, Autres	vegetale	0.0	0.0	0.0	0.
32	Afghanistan	Maïs	vegetale	200000000.0	0.0	21.0	2.
34	Afghanistan	Millet	vegetale	0.0	0.0	3.0	0.
40	Afghanistan	Orge	vegetale	360000000.0	0.0	26.0	2.

In [50]:
```
#Affichage de la proportion d'alimentation animale
print("Proportion d'alimentation animale :", "{:.2f}".format(cereales['Alime
```

```
Proportion d'alimentation animale : 36.14 %
```

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

In [54]:
```
#Création de la colonne proportion par pays
#sous_nutrition_population['Proportion sous-alimentée'] = sous_nutrition_pop
```

In [56]:
```
sous_nutrition_population['Population'] = sous_nutrition_population['Populat
sous_nutrition_population['Nombre en sous-nutrition'] = sous_nutrition_popul
sous_nutrition_population['Proportion sous-alimentée'] = sous_nutrition_popu
```

In [57]:
```
sous_nutrition_population['Proportion sous-alimentée'] = (sous_nutrition_pop
```

In [55]:
```
#sous_nutrition_population[["Pays","Proportion sous-alimentée"]].sort_values
```

Out[55]:

	Pays	Proportion sous-alimentée
84	Inde	1901000.0
136	Pakistan	248000.0
85	Indonésie	236000.0
128	Nigéria	228000.0
16	Bangladesh	215000.0
61	Éthiopie	211000.0
144	Philippines	157000.0
158	République-Unie de Tanzanie	134000.0
157	République populaire démocratique de Corée	120000.0
96	Kenya	119000.0

In [58]:
```
result = sous_nutrition_population[['Pays', 'Proportion sous-alimentée']].so
print(result)
```

```
                                        Pays  Proportion sous-alimentée
84                                      Inde               1.901000e+11
136                                  Pakistan               2.480000e+10
85                                 Indonésie               2.360000e+10
128                                  Nigéria               2.280000e+10
16                                Bangladesh               2.150000e+10
61                                  Éthiopie               2.110000e+10
144                               Philippines               1.570000e+10
158                 République-Unie de Tanzanie               1.340000e+10
157  République populaire démocratique de Corée               1.200000e+10
96                                       Kenya               1.190000e+10
```

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

In [53]:
```
#calcul du total de l'aide alimentaire par pays
aide_par_pays = aide_alimentaire.groupby('Zone')['Aide_alimentaire'].sum()
```

In [55]:
```
aide_par_pays.head()
```

Out[55]:
```
Zone
Afghanistan    185452000
Algérie         81114000
Angola           5014000
Bangladesh     348188000
Bhoutan          2666000
Name: Aide_alimentaire, dtype: int64
```

In [ ]:

In [59]:
```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimen
aide_par_pays = aide_par_pays.sort_values(ascending=False)
aide_par_pays.head(10)
```

Out[59]:
```
Zone
République arabe syrienne           1858943000
Éthiopie                            1381294000
Yémen                               1206484000
Soudan du Sud                        695248000
Soudan                               669784000
Kenya                                552836000
Bangladesh                           348188000
Somalie                              292678000
République démocratique du Congo     288502000
Niger                                276344000
Name: Aide_alimentaire, dtype: int64
```

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

In [61]:
```
#Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis gr
data_filtered = aide_alimentaire[(aide_alimentaire['Année'] >= 2013) & (aide
```

In [66]:
```
data_filtered.head()
```

Out[66]:

	Zone	Année	Produit	Aide_alimentaire
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

In [67]:
```
data_grouped = data_filtered.groupby(['Zone', 'Année'])['Aide_alimentaire'].
```

In [85]:
```
#data_grouped
```

In [72]:
```
#Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de l'
total_food_aid = data_filtered.groupby('Zone')['Aide_alimentaire'].sum().sor
```

In [88]:
```
top_5_countries = total_food_aid.head(5)
top_5_countries
```

Out[88]:
```
Zone
République arabe syrienne    1858943000
Éthiopie                     1381294000
Yémen                        1206484000
Soudan du Sud                 695248000
Soudan                        669784000
Name: Aide_alimentaire, dtype: int64
```

In [75]:
```
#On filtre sur le dataframe avec notre liste
data_top_5 = data_filtered[data_filtered['Zone'].isin(top_5_countries)]
```

In [89]:
```
data_top_5
```

Out[89]:

	Zone	Année	Produit	Aide_alimentaire
354	Éthiopie	2013	Autres non-céréales	170000
355	Éthiopie	2014	Autres non-céréales	466000
356	Éthiopie	2015	Autres non-céréales	244000
357	Éthiopie	2013	Blé et Farin	181066000
358	Éthiopie	2014	Blé et Farin	178646000
...	...	...	...	...
1447	Yémen	2015	Riz, total	1797000
1448	Yémen	2013	Sucre, total	161000
1449	Yémen	2014	Sucre, total	81000
1450	Yémen	2015	Sucre, total	7617000
1451	Yémen	2016	Sucre, total	2675000

155 rows × 4 columns

In [77]:
```
# Affichage des pays avec l'aide alimentaire par année
for year in range(2013, 2017):
    display_year = data_top_5[data_top_5['Année'] == year]
    print(display_year.head())
```

```
         Zone  Année               Produit  Aide_alimentaire
354  Éthiopie   2013   Autres non-céréales            170000
357  Éthiopie   2013          Blé et Farin         181066000
360  Éthiopie   2013          BulgurFarEnt           2939000
362  Éthiopie   2013              Céréales         256196000
365  Éthiopie   2013  Céréales Secondaires          45040000
         Zone  Année               Produit  Aide_alimentaire
355  Éthiopie   2014   Autres non-céréales            466000
358  Éthiopie   2014          Blé et Farin         178646000
361  Éthiopie   2014          BulgurFarEnt           2490000
363  Éthiopie   2014              Céréales         265013000
366  Éthiopie   2014  Céréales Secondaires          70890000
         Zone  Année               Produit  Aide_alimentaire
356  Éthiopie   2015   Autres non-céréales            244000
359  Éthiopie   2015          Blé et Farin          74090000
364  Éthiopie   2015              Céréales          86482000
367  Éthiopie   2015  Céréales Secondaires           7880000
369  Éthiopie   2015    Fruits secs, total            162000
                         Zone  Année                  Produit  \
1008  République arabe syrienne   2016              Blé et Farin
1012  République arabe syrienne   2016              BulgurFarEnt
1016  République arabe syrienne   2016                  Céréales
1026  République arabe syrienne   2016  Légumineuses Sèches,Tot.
1030  République arabe syrienne   2016              Non-céréales

      Aide_alimentaire
1008          11428000
1012           8581000
1016          29569000
1026          21752000
1030          29710000
```

3.9 - Pays avec le moins de disponibilité par habitant

In [78]:
```
#Calcul de la disponibilité en kcal par personne par jour par pays
```

In [90]:
```
dispo_pays_total = dispo_alimentaire[['Zone','Produit','Disponibilité alimen
```

In [91]:
```
dispo_pays_total.sort_values(by="Disponibilité alimentaire (Kcal/personne/jo
```

Out[91]:

Zone	Produit	Disponibilité alimentaire (Kcal/personne/jour)
République centrafricaine	Abats ComestibleAlcool, non ComestibleAliments...	1879.0
Zambie	Abats ComestibleAgrumes, AutresAlcool, non Com...	1924.0
Madagascar	Abats ComestibleAgrumes, AutresAlcool, non Com...	2056.0
Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	2087.0
Haïti	Abats ComestibleAgrumes, AutresAlcool, non Com...	2089.0
République populaire démocratique de Corée	Abats ComestibleAnimaux Aquatiques AutreAvoine...	2093.0
Tchad	Abats ComestibleAlcool, non ComestibleAliments...	2109.0
Zimbabwe	Abats ComestibleAgrumes, AutresAlcool, non Com...	2113.0
Ouganda	Abats ComestibleAgrumes, AutresAlcool, non Com...	2126.0
Timor-Leste	Abats ComestibleAliments pour enfantsAnanasAni...	2129.0

In [95]:
```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
dispo_pays_total.sort_values(by="Disponibilité alimentaire (Kcal/personne/jo
```

Out[95]:

Zone	Produit	Disponibilité alimentaire (Kcal/personne/jour)
République centrafricaine	Abats ComestibleAlcool, non ComestibleAliments...	1879.0
Zambie	Abats ComestibleAgrumes, AutresAlcool, non Com...	1924.0
Madagascar	Abats ComestibleAgrumes, AutresAlcool, non Com...	2056.0
Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	2087.0
Haïti	Abats ComestibleAgrumes, AutresAlcool, non Com...	2089.0
République populaire démocratique de Corée	Abats ComestibleAnimaux Aquatiques AutreAvoine...	2093.0
Tchad	Abats ComestibleAlcool, non ComestibleAliments...	2109.0
Zimbabwe	Abats ComestibleAgrumes, AutresAlcool, non Com...	2113.0
Ouganda	Abats ComestibleAgrumes, AutresAlcool, non Com...	2126.0
Timor-Leste	Abats ComestibleAliments pour enfantsAnanasAni...	2129.0

3.10 - Pays avec le plus de disponibilité par habitant

In [81]:
```
#Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
dispo_pays_total.sort_values(by="Disponibilité alimentaire (Kcal/personne/jo
```

Out[81]:

	Zone	dispo_kcal
1665	Bermudes	51.573455
4250	Dominique	50.938131
4236	Dominique	45.674536
12163	Saint-Kitts-et-Nevis	40.009212
1664	Bermudes	36.126370
1647	Bermudes	34.364661
560	Antigua-et-Barbuda	30.907920
12130	Saint-Kitts-et-Nevis	27.790282
4248	Dominique	27.732278
4237	Dominique	27.455423

3.11 - Exemple de la Thaïlande pour le Manioc

In [98]:
```
sous_nutrition_population.columns
```

Out[98]:
```
Index(['Pays', 'Année_x', 'sous_nutrition', 'Année_y', 'Population',
       'Nombre en sous-nutrition', 'Proportion sous-alimentée'],
      dtype='object')
```

In [100…
```
#création d'un dataframe avec uniquement la Thaïlande
Thaïlande = sous_nutrition_population.loc[sous_nutrition_population['Pays']
Thaïlande.head()
```

Out[100…

	Pays	Année_x	sous_nutrition	Année_y	Population	Nombre en sous-nutrition	Propor s alime
185	Thaïlande	2016-2018	6200000.0	2017	69209810.0	4.291008e+12	620

In [102…
```
#Calcul de la sous nutrition en Thaïlande
print('Proportion  :', "{:.2f}".format(Thaïlande['sous_nutrition'].iloc[0]*1
```

```
Proportion  : 8.96 %
```

In [109…
```
# On calcule la proportion exportée en fonction de la proportion
thai_manioc = dispo_alimentaire.loc[(dispo_alimentaire['Produit'] == "Manioc
print('Proportion de manioc  :', "{:.2f}".format(thai_manioc['Exportations -
                                                                    thai_ma
```

```
Proportion de manioc  : 83.41 %
```

In [ ]: